King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 05

Recursion

Term 261

Reading Assignment

- "Data Structures and Algorithms in Java", 3rd Edition, Adam Drozdek, Cengage Learning, ISBN 978-9814239233
  - Chapter 5 Sections 1-8, 10.
  - Backtracking and the Case Study in Sections 9 and 11 are optional reading.

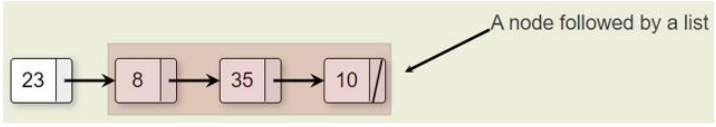
Objectives

- Discuss the following topics:
  - Recursion
  - Recursive Definitions and Data Structures
  - Tracing of Recursive Methods
  - Method Calls and Recursion Implementation
  - Tail and Non-Tail Recursive Methods
  - Direct and Indirect Recursive Methods
  - Nested and non-Nested Recursive Methods
  - Excessive Recursion
  - Final Remarks on Recursion

Recursion

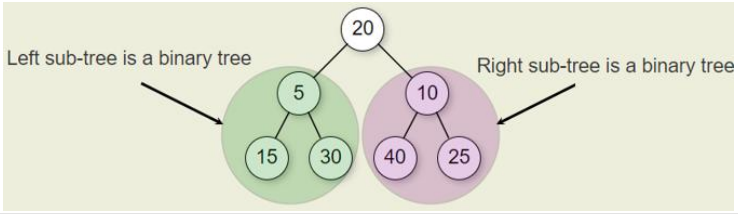
A process of defining a problem, structure, or computation in terms of a smaller instance of itself.

- Recursion is a powerful technique for solving problems which can be defined in terms of smaller versions of itself.
  - break down a problem into one or more simpler sub-problems that are similar in form to the original problem
  - Example: $n! = n * (n - 1)!$
- A recursive data structure is one that:
  - defined in terms of smaller or simpler instances of itself
  - Can be processed recursively.
    - A list can be defined as either (1) an empty list or (2) a node (basic elements) followed by a list.




```
public class Node<T>{
    private T data;
    private Node next;
}
```

    - A binary tree is typically defined as (1) an empty tree or (2) a node pointing to two binary trees




```
public class Node<T> {
   T data;
   Node<T> left;
   Node<T> right;
}
```

What is a Recursive Method?

- A method is recursive if it solves a problem by solving smaller instances of the same problem, typically by calling itself. (i.e., through recursive calls).

Every recursive method must have two key parts:

1. Base Case

- A condition where the recursion **stops**
- Prevents infinite recursion

2. Recursive Case

- A step where the method **calls itself**
- Breaks the problem into **smaller or simpler** instances

```java
public static int factorial(int n) {
    if (n == 0) {
        return 1;  // base case
    } else {
        return n * factorial(n - 1);
    }
}
```

- Example 1: A recursive method for computing n!

$$n! = \begin{cases} 1 & \text{if } n = 0 \quad \text{(base case)} \\ n \times (n-1)! & \text{if } n > 0 \quad \text{(recursive case)} \end{cases}$$

Recursive Methods: Examples

```java
public static long power(int x, int n) {
    // Base case
    if (n == 0) {
        return 1;
    }
    // Recursive case
    return x * power(x, n - 1);
}
```

```java
public static int sumDigits(int n) {
    if (n == 0) {
        return 0; // base case
    } else {
        return (n % 10) + sumDigits(n / 10);
    }
}
```

```java
public static int findMax(int[] arr, int index) {
    if (index == arr.length - 1) {
        return arr[index]; // Base case: last ele
    }
    int maxRest = findMax(arr, index + 1); // Rec
    return Math.max(arr[index], maxRest);
}
```

```java
public static boolean isPalindrome(String s) {
    if (s.length() <= 1) {
        return true; // Base case: empty or single chara
    }
    if (s.charAt(0) != s.charAt(s.length() - 1)) {
        return false;
    }
    return isPalindrome(s.substring(1, s.length() - 1));
}
```

```java
static boolean isPalindrome(String s, int left, int right) {
    // Base case: crossed or met in the middle
    if (left >= right)
        return true;

    // If mismatch → not a palindrome
    if (s.charAt(left) != s.charAt(right))
        return false;

    // Recurse inward
    return isPalindrome(s, left + 1, right - 1);
}
```

Recursive Methods: Examples

```java
public static int countX(int[] arr, int x) {
    return countX(arr, arr.length - 1, x);
}

private static int countX(int[] arr, int index, int x) {
    if (index < 0) {
        return 0;
    } else if (arr[index] == x) {
        return 1 + countX(arr, index - 1, x);
    } else {
        return countX(arr, index - 1, x);
    }
}
```

```java
public static int countX(int[] arr, int x) {
    return countX(arr, 0, x); // start from first element
}

// Recursive method with index parameter
private static int countX(int[] arr, int index, int x) {
    if (index >= arr.length) {
        return 0;
    } else if (arr[index] == x) {
        return 1 + countX(arr, index + 1, x);
    } else {
        return countX(arr, index + 1, x);
    }
```

```java
static int countX(int[] a, int x) {
    // Base case: empty array
    if (a.length == 0)
        return 0;

    // Split into first element and the rest
    int first = a[0];
    int[] rest = Arrays.copyOfRange(a, 1, a.length);

    // Recursive case with explicit +1 or +0
    if (first == x)
        return 1 + countX(rest, x);
    else
        return 0 + countX(rest, x);
}
```

Value	1	2	0	1	1

Index	0	1	2	3	4

Binary Search (Recursive)

```java
public static int binarySearch(int[] arr, int target, int low, int high) {
    if (low > high) {
        return -1; // Base case: not found
    }

    int mid = (low + high) / 2;

    if (arr[mid] == target) {
        return mid;
    } else if (target < arr[mid]) {
        return binarySearch(arr, target, low, mid - 1);
    } else {
        return binarySearch(arr, target, mid + 1, high);
    }
}
```

Linear and Binary Recursion

- Linear Recursion:
  - a method makes at most one recursive call each time it is invoked
  - useful when we view a problem in terms of a first or last element plus a remaining set
- Binary Recursion:
  - a method makes two recursive calls each time it is invoked (two subproblems)

```java
public static int countX(int[] arr, int x) {
    return countX(arr, 0, x); // start from first element
}

// Recursive method with index parameter
private static int countX(int[] arr, int index, int x) {
    if (index >= arr.length) {
        return 0;
    } else if (arr[index] == x) {
        return 1 + countX(arr, index + 1, x);
    } else {
        return countX(arr, index + 1, x);
    }
```

```java
public static int countX(int[] arr, int x) {
    return countX(arr, x, 0, arr.length - 1);
}

private static int countX(int[] arr, int x, int left, int right) {
    // Base case: empty range
    if (left > right) {
        return 0;
    }

    // Base case: one element
    if (left == right) {
        return arr[left] == x ? 1 : 0;
    }

    int mid = (left + right) / 2;

    // Binary recursion: split into two halves
    int leftCount = countX(arr, x, left, mid);
    int rightCount = countX(arr, x, mid + 1, right);

    return leftCount + rightCount;
}
```

Recursive Methods for Linked List

```java
public static void printList(Node head) {
    if (head == null) {
        return;  // base case
    }
    System.out.print(head.data + " ");
    printList(head.next);  // recursive step
}
```



```java
public static void printList(Node head) {
    if (head != null) {
        System.out.print(head.data + " ");
        printList(head.next);
    }
}
```

```java
public static int findMin(Node head) {
    if (head.next == null) {
        return head.data;
    }

    int minInRest = findMin(head.next);
    return Math.min(head.data, minInRest);
}
```

```java
public static boolean search(Node head, int target) {
    if (head == null) {
        return false; // base case: reached end of list
    }

    if (head.data == target) {
        return true; // found the target
    }

    return search(head.next, target); // recursive call
}
```

```java
// Recursive method to calculate size
public static int getSize(Node head) {
    if (head == null) {
        return 0;
    }
    return 1 + getSize(head.next);
}
```

The need for Auxiliary (or Helper) Methods

- Auxiliary or helper methods are used for one or more of the following reasons:
  - To make recursive methods more efficient.
  - To make the user interface to a method simpler by hiding the method's initializations.
- Example 1: Consider the method:

```
public long factorial(int x) {
    if (x < 0)
        throw new IllegalArgumentException("Negative argument");
    else if (x == 0)
        return 1;
    else
        return x * factorial(x - 1);
}
```

```
public long factorial(int x){
   if (x < 0)
      throw new IllegalArgumentException("Negative argument");
   else
      return factorialAuxiliary(x);
}
```

```
private long factorialAuxiliary(int x){
   if (x == 0)
      return 1;
   else
      return x * factorialAuxiliary(x - 1);
}
```

- The condition x < 0 which should be executed only once is executed in each recursive call. We can use a private auxiliary method to avoid this

The need for Auxiliary (or Helper) Methods…

- Example 2: Consider the method:

```java
public static int binarySearch(int[] arr, int target, int low, int high) {
    if (low > high) {
        return -1; // Base case: not found
    }

    int mid = (low + high) / 2;

    if (arr[mid] == target) {
        return mid;
    } else if (target < arr[mid]) {
        return binarySearch(arr, target, low, mid - 1);
    } else {
        return binarySearch(arr, target, mid + 1, high);
    }
}
```

- The first time the method is called, the parameter low and high must be set to 0 and array.length – 1 respectively. Example:

```java
int result = binarySearch (array, target ,0, array.length -1);
```

- From a user's perspective, the parameters low and high introduce an unnecessary complexity that can be avoided by using an auxiliary method:

The need for Auxiliary (or Helper) Methods…

- A call to the method becomes simple:

```
int result = binarySearch(array, target);
```

```java
public int binarySearch(int[] array, int target) {
    return binarySearch(array, target, 0, array.length - 1);
}

private int binarySearch(int[] array, int target, int low, int high) {
    if (low > high)
        return -1;

    int middle = (low + high) / 2;

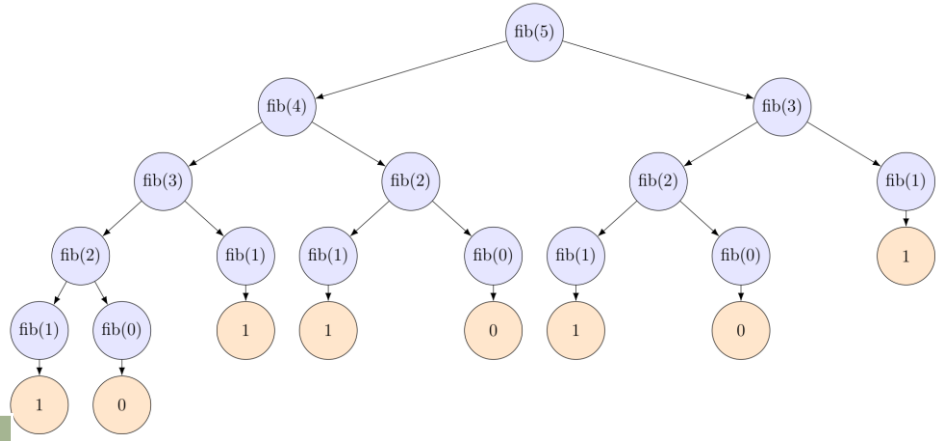
    if (array[middle] == target)
        return middle;
    else if (array[middle] < target)
        return binarySearch(array, target, middle + 1, high);
    else
        return binarySearch(array, target, low, middle - 1);
}
```

Tracing of Recursive Methods: Example 1

- A **recursion tree** is a very visual and intuitive way to trace recursive methods. It is a **visual representation** of the **recursive calls** made by a function during its execution
  - The root is the initial call.
  - Each node is a recursive call made by its parent.
  - Leaves represent base cases where recursion stops

$$\text{fib}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & \text{if } n \geq 2 \end{cases}$$

```java
public static int fibonacci(int n) {
    if (n == 0) {
        return 0; // Base case
    } else if (n == 1) {
        return 1; // Base case
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```



0	1	2	3	4	5	6	7	8	9
0	1	1	2	3	5	8	13	21	34

Tracing of Recursive Methods: Example 2

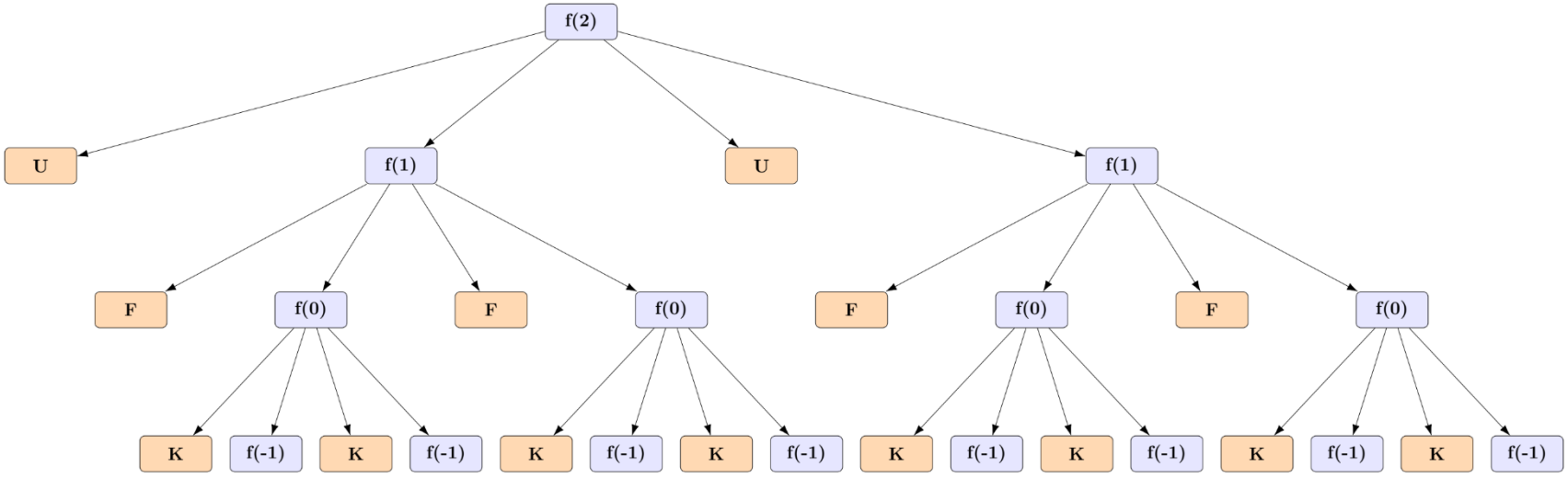
- Consider the recursive method f defined below. Draw the recursive tree generated by the call f("KFU", 2) and hence determine the total number of method calls and the output of the following program:

```
public static void main(String[] args) {
    f("KFU", 2);
}

public static void f(String s, int index) {
    if (index >= 0) {
        System.out.print(s.charAt(index));
        f(s, index - 1);
        System.out.print(s.charAt(index));
        f(s, index - 1);
    }
}
```

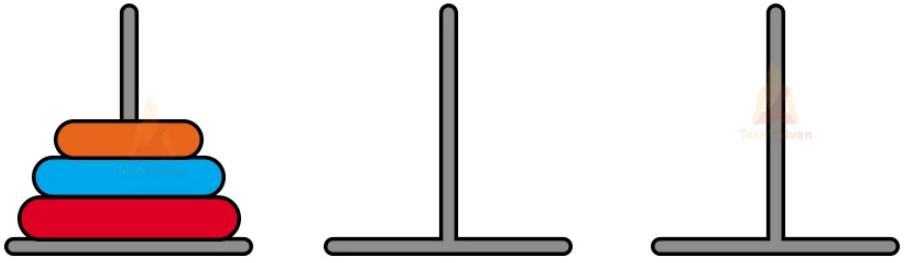
The output is: UFKKFKKUFKKFKK

The total number of method calls is 15



Tracing of Recursive Methods: Example 3

- The Towers of Hanoi problem
  - A total of n disks are arranged on a peg A from the largest to the smallest; such that the smallest is at the top. Two empty pegs B and C are provided.
  - It is required to move the n disks from peg A to peg C under the following restrictions:
    - Only one disk may be moved at a time.
    - A larger disk must not be placed on a smaller disk.
    - In the process, any of the three pegs may be used as temporary storage.

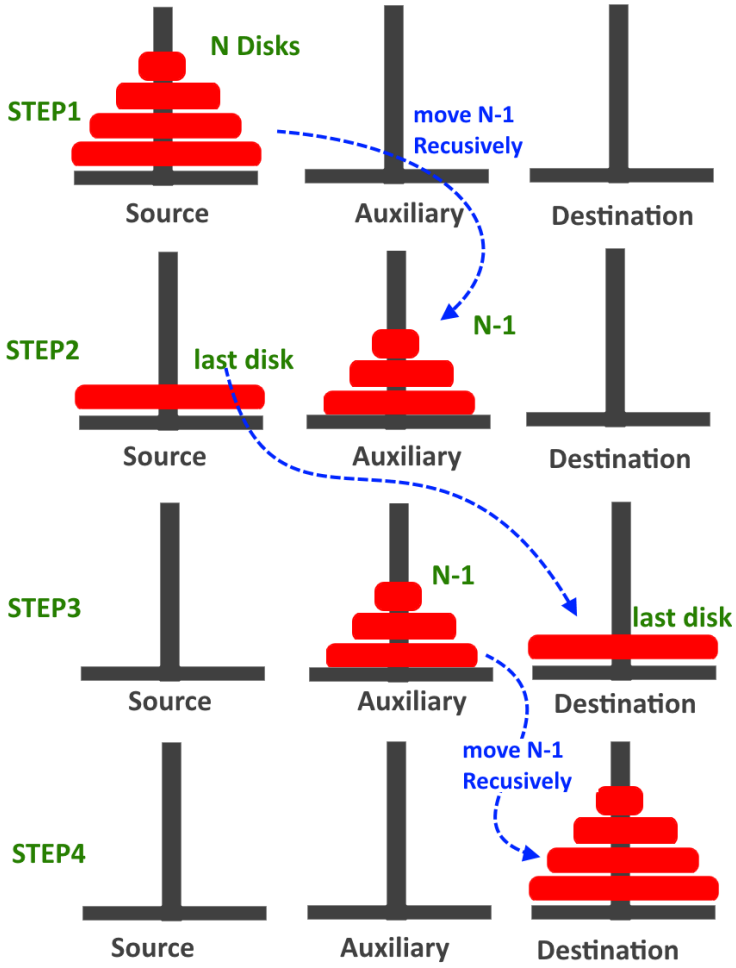


Tracing of Recursive Methods

- Suppose we can solve the problem for n – 1 disks. Then to solve for n disks use the following algorithm:
  - Move n – 1 disks from peg A to peg B
  - Move the nth disk from peg A to peg C
  - Move n – 1 disks from peg B to peg C

```
public static void main(String[] args) {
   int n = 3;
   hanoi(n, 'A', 'C', 'B');
}

public static void hanoi(int n, char from, char to, char temp) {
   if (n == 1)
      System.out.println(from + " --------> " + to);
   else {
      hanoi(n - 1, from, temp, to);      // Step 1: Move n-1 disks fro
      System.out.println(from + " --------> " + to); // Step 2: Move t
      hanoi(n - 1, temp, to, from);      // Step 3: Move n-1 disks fro
   }
}
```



Tracing of Recursive Methods

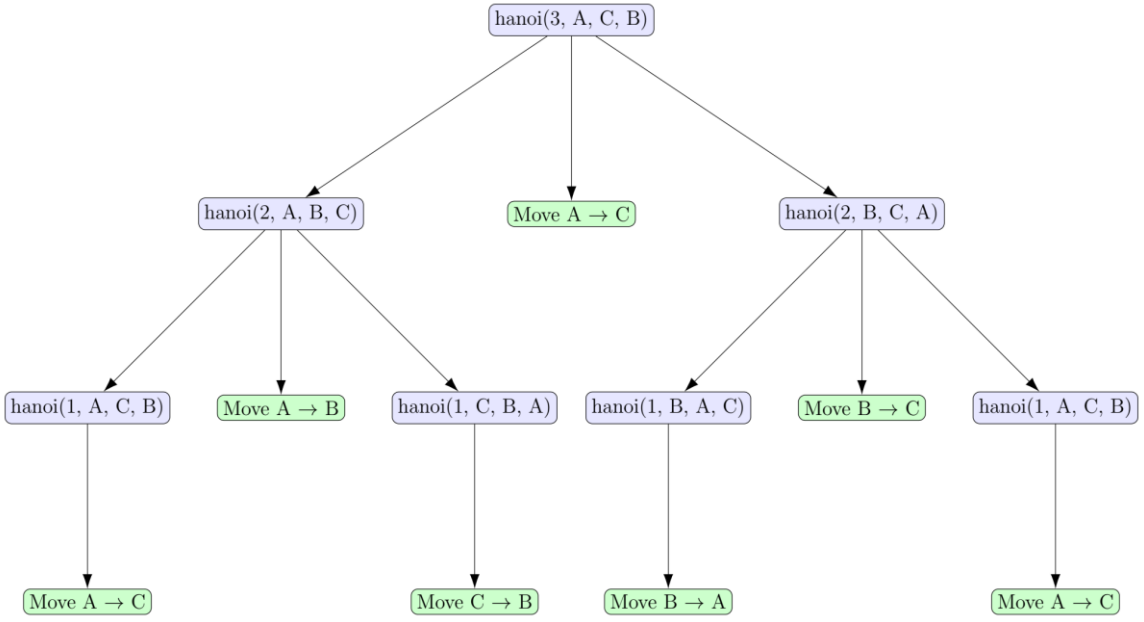
- Draw the recursion tree of the method hanoi for n = 3 and hence determine the output of the above program.

```
public static void hanoi(int n, char from, char to, char temp) {
   if (n == 1)
      System.out.println(from + " --------> " + to);
   else {
      hanoi(n - 1, from, temp, to);      // Step 1: Move n-1 disks
      System.out.println(from + " --------> " + to); // Step 2: Mov
      hanoi(n - 1, temp, to, from);      // Step 3: Move n-1 disks
   }
}
```

hanoi(3, 'A', 'C', 'B')

output of the program is:

A → C
A → B
C → B
A → C
B → A
B → C
A → C



#moves=2^n-1

Method Calls and Recursion Implementation

- When a method is called an Activation Record (AR) is created and pushed onto the stack. It stores:
  - Parameters → the values passed into the method.
  - Local variables → the variables declared inside the method.
  - Return address → the statement right after the method call
  - Previous AR address → so the stack can "unwind" correctly
  - Return value address → where the method's result will be placed

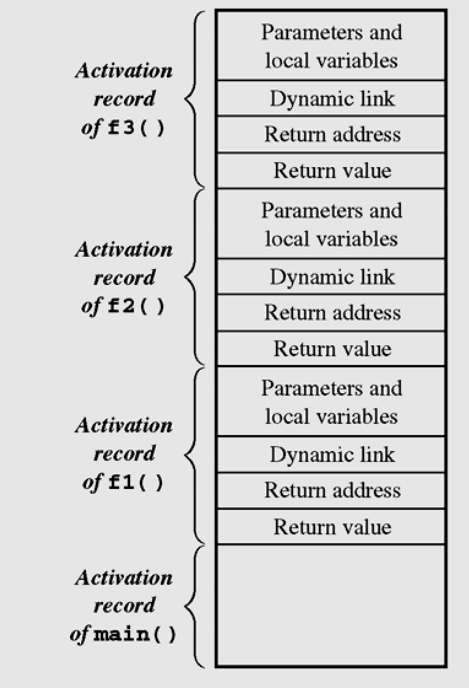


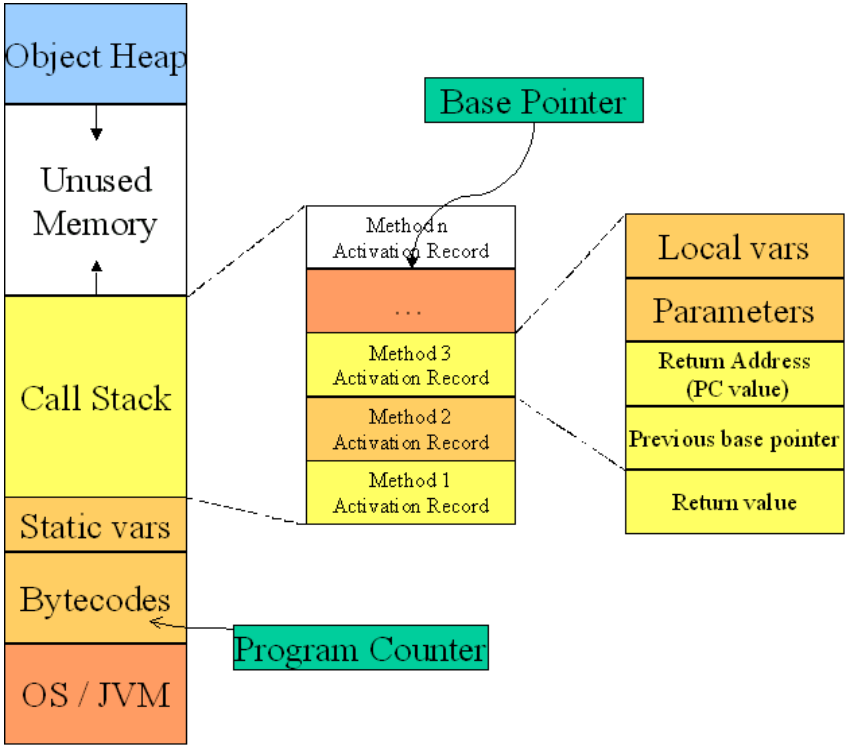
Figure 5-1 Contents of the run-time stack when main() calls method f1(), f1() calls f2(), and f2() calls f3()

- When a method finishes and returns:
  - The return value is passed to the caller, and the activation record is then removed from the stack.

- Recursion is handled in a similar way.
  - Each recursive call creates a separate Activation Record.
  - As each recursive call completes, its Activation Record is popped from the stack. Ultimately control passes back to the calling statement.

Method Calls and Recursion Implementation

- Modern computers use a stack as the primary memory management model for running programs.
- Each running program has its own memory allocation containing the typical layout as shown below.



Direct and Indirect Recursive Methods

- A method is directly recursive if it contains an explicit call to itself.

```
public static int factorial(int n) {
    if (n == 0) {
        return 1;  // base case
    } else {
        return n * factorial(n - 1);
    }
}
```

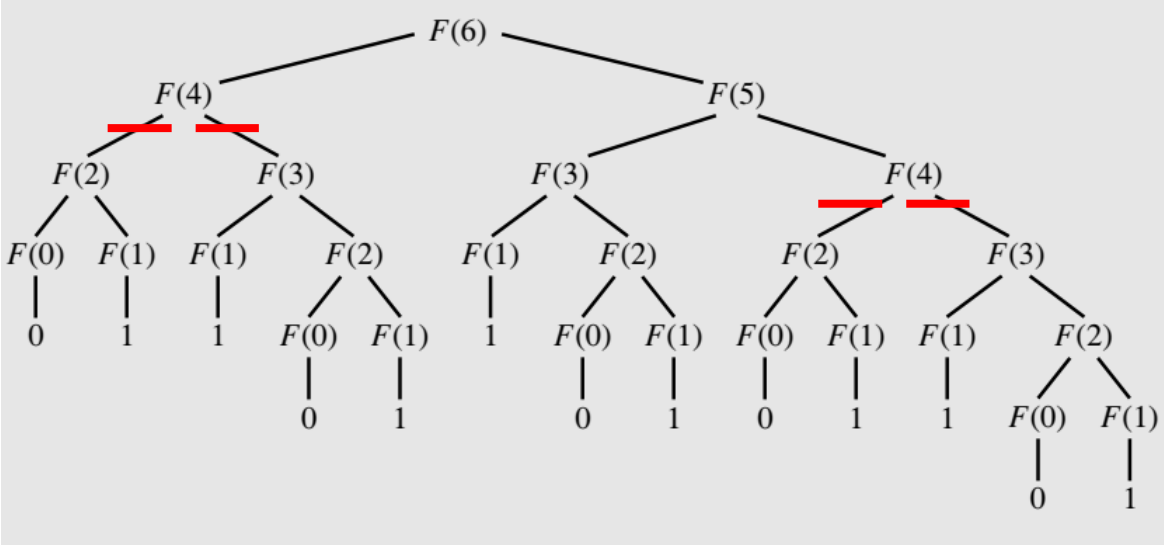
- A method x is indirectly recursive if it contains a call to another method which in turn calls x. They are also known as mutually recursive methods:

```
public static boolean isEven(int n) {
    n = Math.abs(n); // handle negative numbers
    if (n == 0) {
        return true;
    } else {
        return isOdd(n - 1);
    }
}

public static boolean isOdd(int n) {
    return !isEven(n);
}
```

Excessive Recursion

- A recursive method is excessively recursive if it repeats computations for some parameter values.
- Example: The call fib(6) results in two repetitions of f(4). This in turn results in repetitions of fib(3), fib(2), fib(1) and fib(0):




```java
public static int fibonacci(int n) {
    if (n == 0) {
        return 0; // Base case
    } else if (n == 1) {
        return 1; // Base case
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Fibonacci Numbers

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

The definition states that if the first two numbers are 0 and 1, then any number in the sequence is the sum of its two predecessors. But these predecessors are in turn sums of their predecessors, and so on.

Excessive Recursion

```java
public static int fibonacci(int n) {
    if (n == 0) {
        return 0; // Base case
    } else if (n == 1) {
        return 1; // Base case
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Iterative algorithm Recursive algorithm

n	Number of Additions	Number of Additions	Number of Calls
6	5	12	25
10	9	88	177
15	14	986	1,973
20	19	10,945	21,891
25	24	121,392	242,785
30	29	1,346,268	2,692,537

```java
public static int iterativeFib(int n) {
    if (n < 2) {
        return n;  // base cases: fib(0)
    }

    int last = 0;
    int current = 1;

    for (int i = 2; i <= n; i++) {
        int temp = current;
        current = current + last;
        last = temp;
    }

    return current;
}
```

Tail and Non-Tail Recursive Methods

the recursive call is the last operation performed by the function.

- A method is tail recursive if it executes one recursive call in each recursive case and if there are **NO** pending statements after that call.
- Example 1:

```
public static void print(int n) {
    if (n == 0) return;
    System.out.println(n);
    print(n - 1);
}
```

```
public static void f1(int n){
   System.out.print(n + "  ");
   if(n > 0)
      f1(n - 1);
}
```

- Example 2:

```
public static void printReverse(int n) {
    if (n == 0) return;
    printReverse(n - 1);
    System.out.println(n);
}
```

```
public static void f3(int n){
   if(n > 6){
      System.out.print(2*n + "  ");
      f3(n - 2);
   } else if(n > 0){
      System.out.print(n + "  ");
      f3(n - 1);
   }
}
```

Tail and Non-Tail Recursive Methods

Call myself, and I'm done.

- A recursive method is non-tail recursive if, in at least one of its recursive cases, at least one of the following holds:
  1. A recursive call is followed by another statement
  2. It has a recursive call that is part of an expression
  3. It has more than one recursive call executed in each call.
     - e.g. Hanoi:

1

```
public static void f4(int n){
   if (n > 0)
      f4(n - 1);
   System.out.print(n + "   ");
}
```

2

```
long factorial(int x) {
    if (x == 0)
        return 1;
    else
        return x * factorial(x - 1);
}
```

- Example 1:
  - After each recursive call there is a pending
  - System.out.print(n + " ") operation.

- Example 2:
  - After each recursive call there is a pending * operation.

3

```
public static void hanoi(int n, char from, char to, char temp) {
   if (n == 1)
      System.out.println(from + " --------> " + to);
   else {
      hanoi(n - 1, from, temp, to);      // Step 1: Move n-1 disks
      System.out.println(from + " --------> " + to); // Step 2: Mo
      hanoi(n - 1, temp, to, from);      // Step 3: Move n-1 disks
   }
}
```

Why tail recursion? Tail vs Non-Tail

Call myself, then do something with the result.

- It is desirable to have tail-recursive methods, because:
  - **Memory efficiency**: Tail recursion avoids consuming more stack space with each call.
    - The number of stack frames does **not** grow with the number of recursive calls
    - Can use O(1) stack space with optimization
  - **Prevents stack overflow**: Deep recursive calls won't crash the program.

- Tail recursion is important in languages like Prolog and functional languages (Haskell, Clean, Miranda, SML) that lack explicit loop constructs (loops are simulated by recursion).

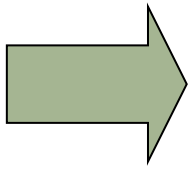
```
int factoria(int n) {
    if (n == 0) return 1;
    return n *factorialTail(n - 1,);
}
```

```
int factorialTail(int n, int f) {
    if (n == 0) return f;
    return factorialTail(n - 1, f * n);
}
```

Converting a tail-recursive method to an iterative method

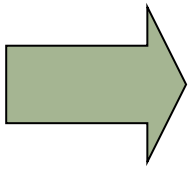
- It is easy to convert a tail recursive method into an iterative one
- Since tail recursion doesn't need to remember pending operations, you can directly convert it into a loop.

```
public static void f1(int n){
   System.out.print(n + "  ");
   if(n > 0)
      f1(n - 1);
}
```



```
public static void f1(int n) {
    for (int k = n; k >= 0; k--) {
        System.out.print(k + "  ");
    }
}
```

```
public static void f3(int n){
   if(n > 6){
      System.out.print(2*n + "  ");
      f3(n - 2);
   } else if(n > 0){
      System.out.print(n + "  ");
      f3(n - 1);
   }
}
```



```
public static void f3(int n) {
    while (n > 0) {
        if (n > 6) {
            System.out.print(2 * n + "  ");
            n = n - 2;
        } else if (n > 0) {
            System.out.print(n + "  ");
            n = n - 1;
        }
    }
}
```

Tail recursive method

Corresponding iterative method

Common Errors in Writing Recursive Methods

- The method does not call itself directly or indirectly.
- Non-terminating Recursive Methods (Infinite recursion):
  - No base case.

```
int badFactorial(int x) {
    return x * badFactorial(x-1);
}
```

  - The base case is never reached for some parameter values.

```
int anotherBadFactorial(int x) {
   if (x == 0)
      return 1;
   else
      return x * (x - 1) * anotherBadFactorial(x - 2);
   // When x is odd, we never reach the base case!!
}
```

Common Errors in Writing Recursive Methods…

- Post increment and decrement operators must not be used since the update will not occur until AFTER the method call - infinite recursion.

```
public static int sumArray (int[] x, int index) {
   if (index == x.length) return 0;
   else
      return x[index] + sumArray(x, index++);
}
```

- Local variables must not be used to accumulate the result of a recursive method. Each recursive call has its own copy of local variables.

```
public static int sumArray(int[] x, int index) {
    int sum = 0;
    if (index == x.length) {
        return sum;
    } else {
        sum += x[index];
        return sumArray(x, index + 1);
    }
}
```

Common Errors in Writing Recursive Methods…

- The use of instance or static variables in recursive methods should be avoided.
  - Although it is not an error, it is bad programming practice. These variables may be modified by code outside the method and cause the recursive method to return a wrong result.

```
public class Sum {
   private int sum;

   public int sumArray(int[] x, int index) {
      if (index == x.length)
         return sum;
      else {
         sum += x[index];
         return sumArray(x, index + 1);
      }
   }
}
```

```
public class Sum {

    public int sumArray(int[] x, int index, int sum) {
        if (index == x.length)
            return sum;
        else
            return sumArray(x, index + 1, sum + x[index]);
    }
}
```

Solution: Sum is a parameter

Recursion vs Loops

Aspect	Recursion	Loops
What is it?	• A function calls itself to solve smaller instances until a base case is reached.	• Repeats a block of code until a condition is met.
Control Structure	• Repeated function calls	• Loops (for, while)
Efficiency	- Uses more memory due to call stack - Might rise stack overflow if too deep - Have greater time requirements (processing stack) - Can be inefficient if not optimized (e.g., naive Fibonacci)	- More memory efficient (constant memory space) - Generally faster (no function call overhead)
When preferred?	- Elegant for naturally recursive problems (trees) - Good for breaking down complex problems - Simplifies divide-and-conquer algorithms (merge sort, quick sort)	- Linear processing over arrays/lists